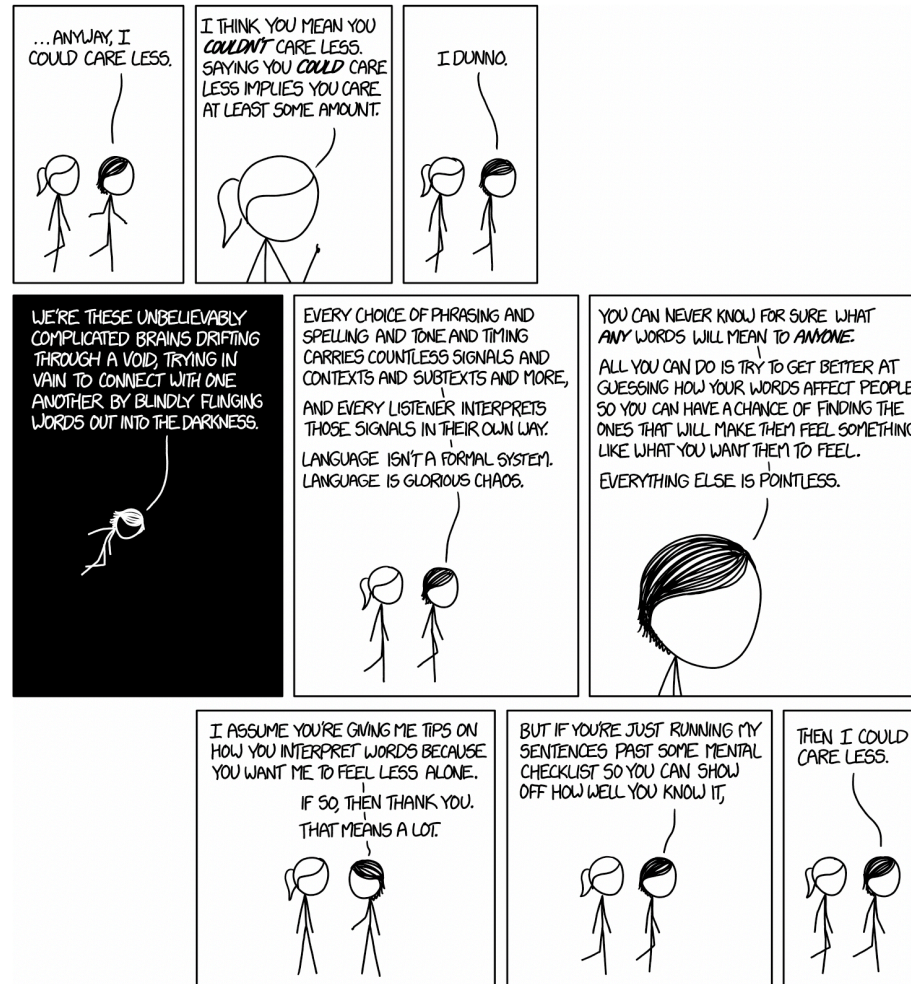


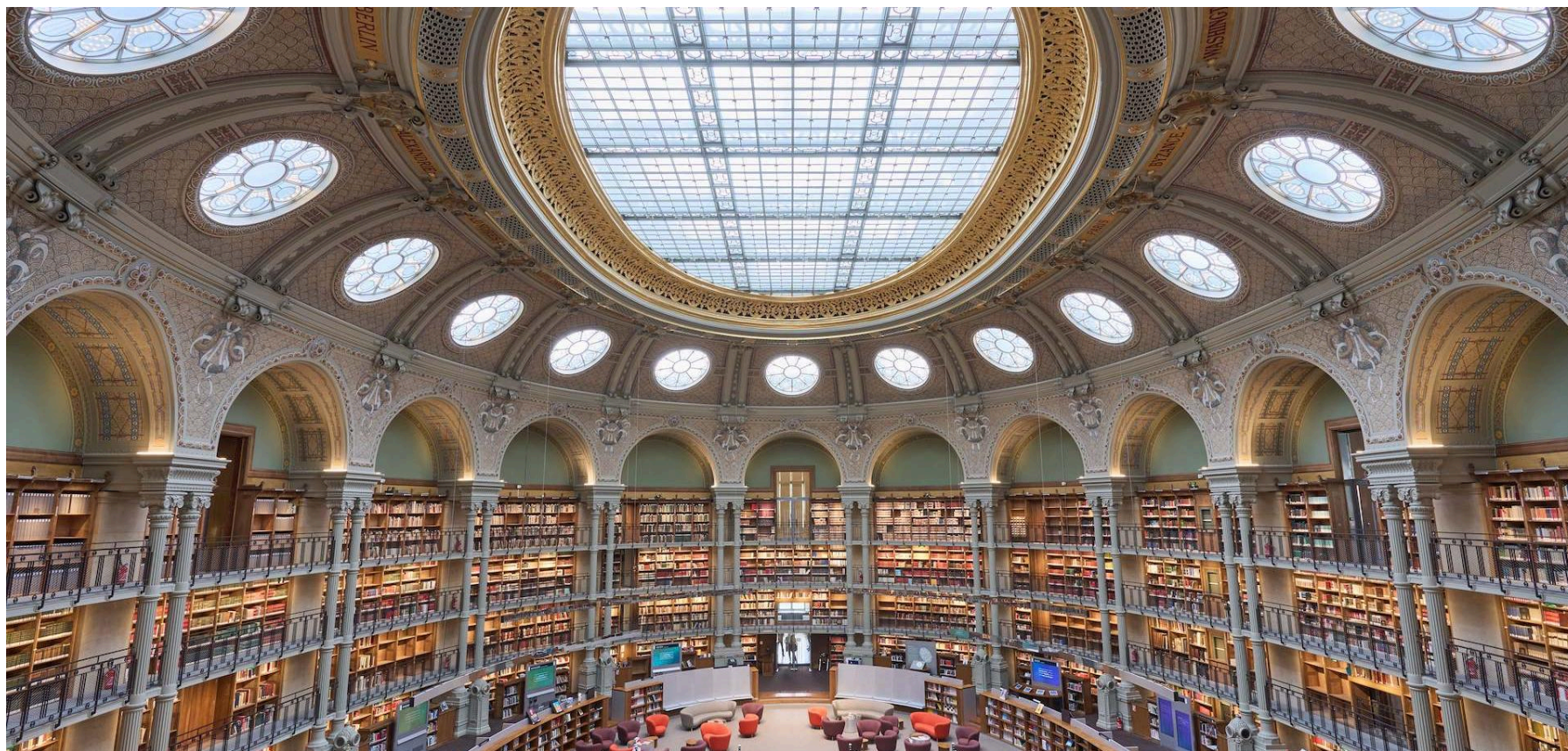
# **Course 1: Introduction & Recap**

**Welcome to the Advanced NLP  
course!**

# Human Language



# Data





# Datasources for NLP



1

Allez sur [wooclap.com](https://wooclap.com)

2

Entrez le code d'événement dans  
le bandeau supérieur

Code d'événement  
**ITIQPZ**



Activer les réponses par SMS

# But also ..

## Data Sources for NLP



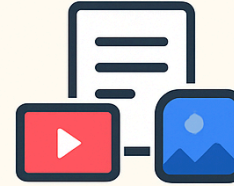
Web &  
Social Media



Conversations



Documents



Speech &  
Subtitles



Multimodal



Human  
Contributions

# What is NLP?



1

Allez sur [wooclap.com](https://wooclap.com)

2

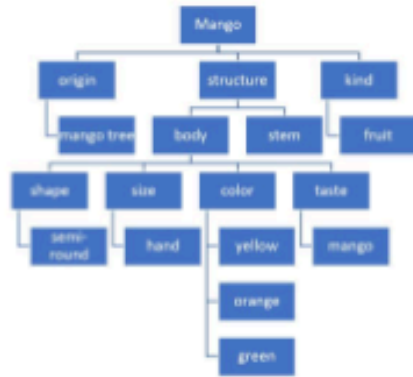
Entrez le code d'événement dans  
le bandeau supérieur

Code d'événement  
**ITIQPZ**



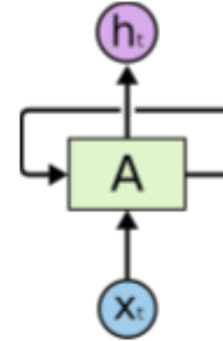
Activer les réponses par SMS

# NLP in recent years

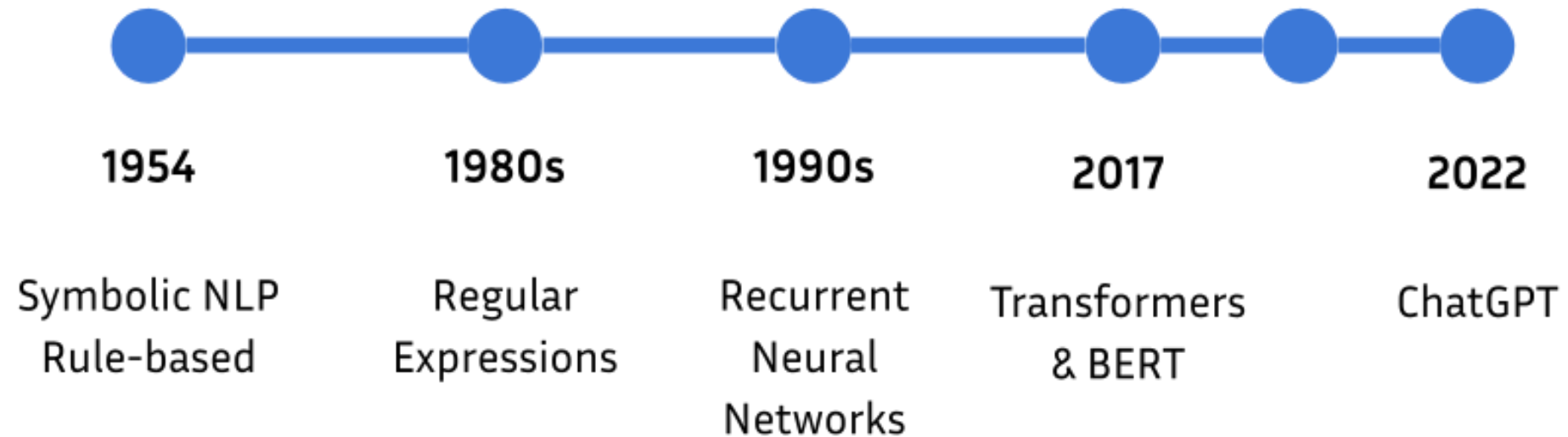


example@gmail.com

`@[[a-zA-Z0-9_+]+].[[a-zA-Z0-9_+]+]`



GPT-3  
2020





# NLP in 2025

How to write a course for advanced NLP?

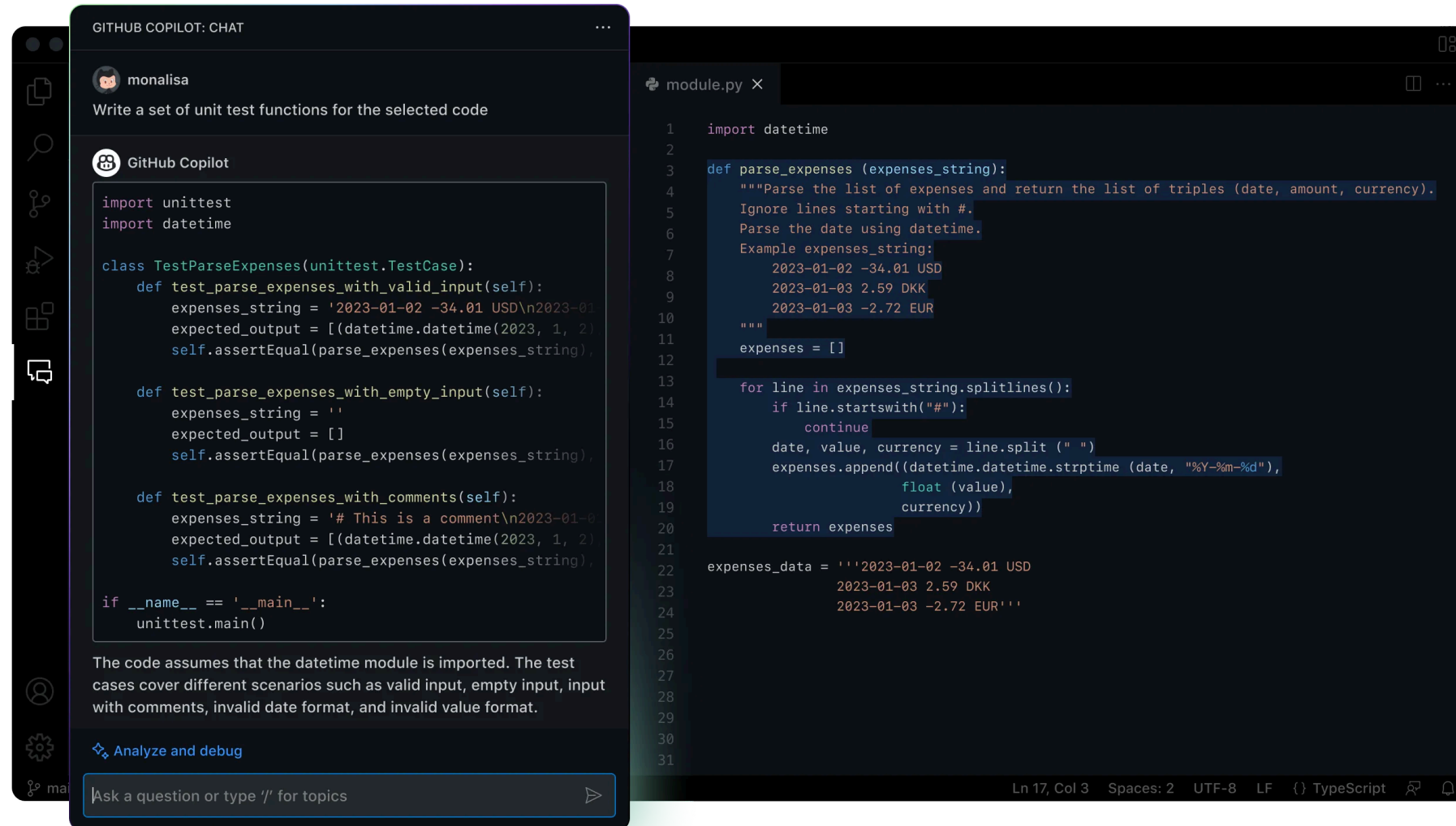


Writing a course for **advanced NLP (Natural Language Processing)** involves a thoughtful balance of theory, practical applications, and emerging trends in the field. You will need to structure the course to cover both foundational knowledge and the latest advancements in NLP. Below is a guide to help you develop a comprehensive and effective course.

## Step 1: Define Course Objectives

Start by identifying the key objectives for the course. For example, students should be able to:

# NLP in 2025



The image shows a GitHub Copilot Chat window on the left and a code editor on the right. The chat window displays a conversation with a user named 'monalisa' who asks for unit test functions. GitHub Copilot responds with a Python script that includes imports for 'unittest' and 'datetime', a class 'TestParseExpenses' with three test methods, and a main block. Below the code, a note explains the assumptions and test cases. At the bottom, there is a button to 'Analyze and debug' and a text input field for asking questions.

GITHUB COPILOT: CHAT

monalisa

Write a set of unit test functions for the selected code

GitHub Copilot

```
import unittest
import datetime

class TestParseExpenses(unittest.TestCase):
    def test_parse_expenses_with_valid_input(self):
        expenses_string = '2023-01-02 -34.01 USD\n2023-01-03 2.59 DKK\n2023-01-03 -2.72 EUR'
        expected_output = [(datetime.datetime(2023, 1, 2), -34.01, 'USD'), (datetime.datetime(2023, 1, 3), 2.59, 'DKK'), (datetime.datetime(2023, 1, 3), -2.72, 'EUR')]
        self.assertEqual(parse_expenses(expenses_string), expected_output)

    def test_parse_expenses_with_empty_input(self):
        expenses_string = ''
        expected_output = []
        self.assertEqual(parse_expenses(expenses_string), expected_output)

    def test_parse_expenses_with_comments(self):
        expenses_string = '# This is a comment\n2023-01-02 -34.01 USD\n2023-01-03 2.59 DKK\n2023-01-03 -2.72 EUR'
        expected_output = [(datetime.datetime(2023, 1, 2), -34.01, 'USD'), (datetime.datetime(2023, 1, 3), 2.59, 'DKK'), (datetime.datetime(2023, 1, 3), -2.72, 'EUR')]
        self.assertEqual(parse_expenses(expenses_string), expected_output)

if __name__ == '__main__':
    unittest.main()
```

The code assumes that the datetime module is imported. The test cases cover different scenarios such as valid input, empty input, input with comments, invalid date format, and invalid value format.

Analyze and debug

Ask a question or type '/' for topics

module.py

```
1 import datetime
2
3 def parse_expenses(expenses_string):
4     """Parse the list of expenses and return the list of triples (date, amount, currency).
5     Ignore lines starting with #.
6     Parse the date using datetime.
7     Example expenses_string:
8         2023-01-02 -34.01 USD
9         2023-01-03 2.59 DKK
10        2023-01-03 -2.72 EUR
11     """
12     expenses = []
13
14     for line in expenses_string.splitlines():
15         if line.startswith("#"):
16             continue
17         date, value, currency = line.split(" ")
18         expenses.append((datetime.datetime.strptime(date, "%Y-%m-%d"),
19                         float(value),
20                         currency))
21     return expenses
22
23 expenses_data = '''2023-01-02 -34.01 USD
24                  2023-01-03 2.59 DKK
25                  2023-01-03 -2.72 EUR'''
26
27
28
29
30
31
```

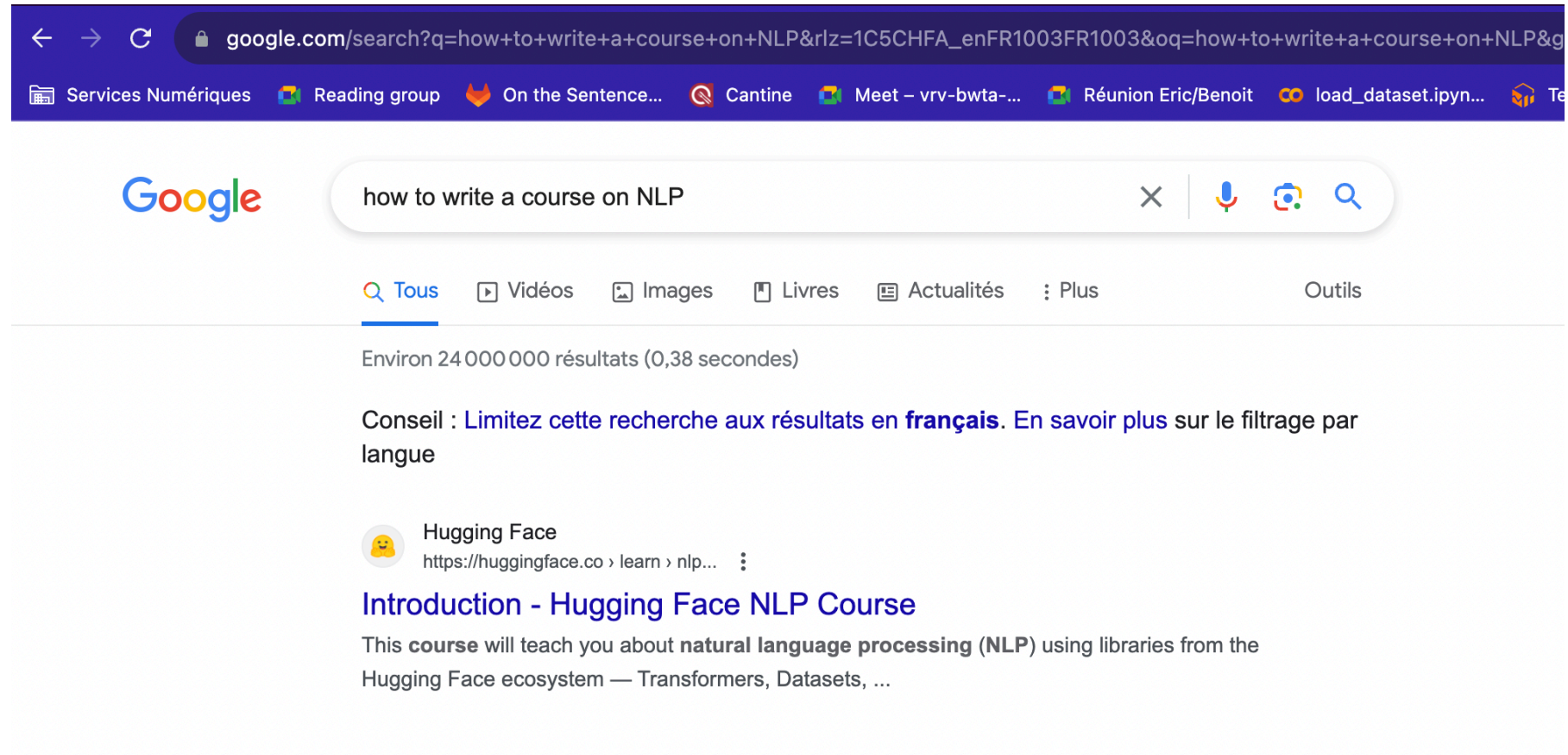
Ln 17, Col 3 Spaces: 2 UTF-8 LF {} TypeScript

# NLP in 2025

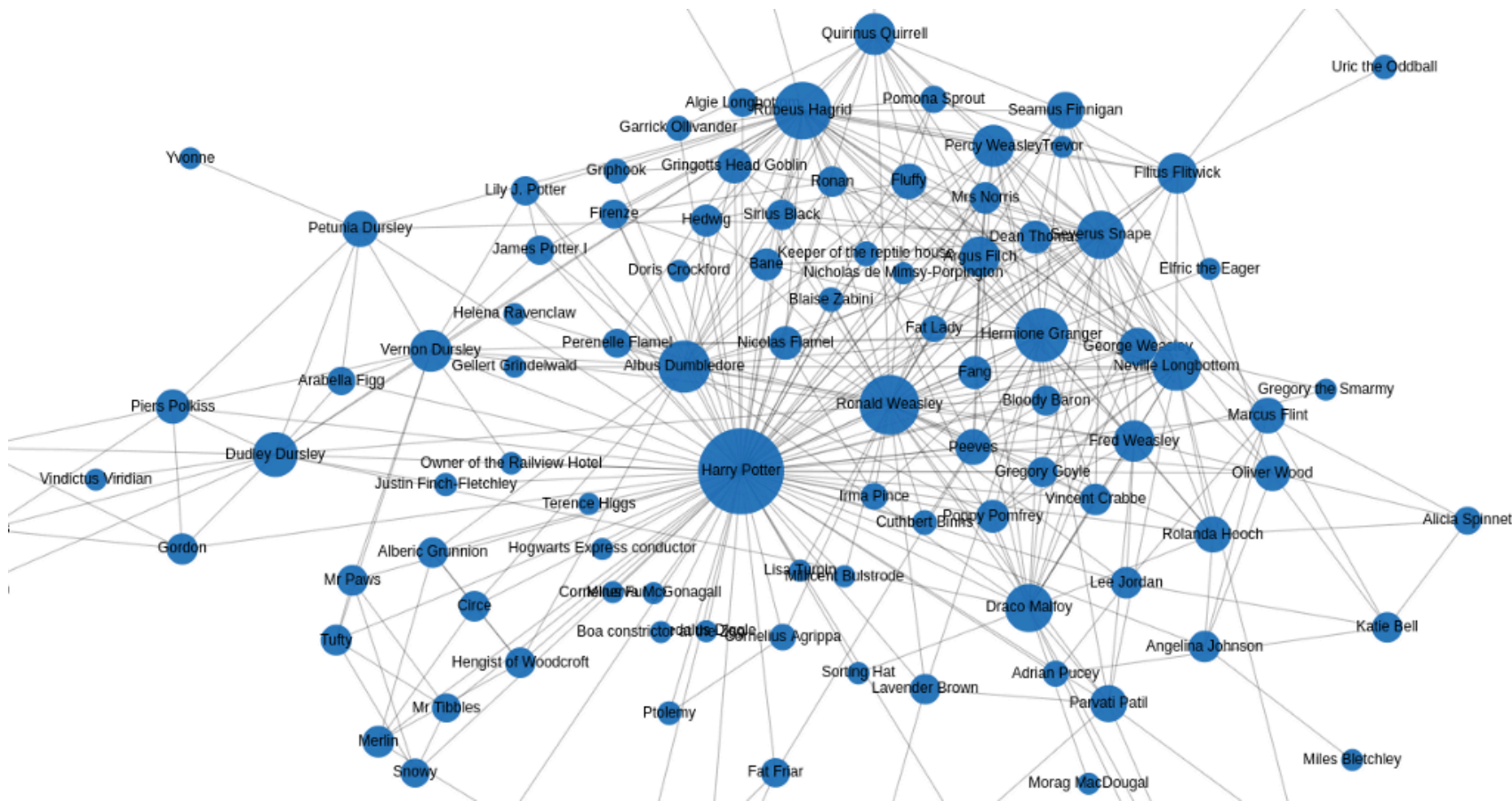
“ *A picture of the computer science school EPITA, with the school logo on a banner.* ”



# NLP in 2025



# NLP in 2025





# NLP perspectives





**So, is NLP solved?**  
**(No.)**

# NLP challenges



Give me a random string of length 35



Sure, here's a random string of length 35:



``J9kPzHfRq81G2sT7Nv6yX1W4c00mZoBwA5U3j``

>



Can you invert all letters in the following sentence: "I am currently giving an interesting course in Natural Language Processing at EPITA in Le-Kremlin-Bicêtre."



Certainly! Here's the sentence with all the letters inverted:



"I ma yltnerurc ygivni na gnitseretni esruoc ni lartuaN egaugnal gnicessorP ta ATIPE ni ertnebiC-tciB-nimeIK-eL."

# NLP challenges



It takes 3 hours to dry 3 towels. How long does it take to dry 9 towels?



If it takes 3 hours to dry 3 towels, it implies that you can dry 1 towel in 3 hours. To find out how long it takes to dry 9 towels, you can use the same rate:   

1 towel takes 3 hours to dry, so:

9 towels will take 9 times as long, which is:

9 towels x 3 hours/towel = 27 hours.

So, it will take 27 hours to dry 9 towels.

# NLP challenges



Let's picture the CEO of a very big international company. The CEO has to do a lot of business trips all around the world, so the CEO's wife is alone to help the children with the homework. Write the informal text message of the wife where she expresses her anger.



Sure, here's an informal text message where the CEO's wife expresses her frustration:

"Ugh, this is getting ridiculous! 🤔 Another week alone with the kids while he's jet-setting around the world. 🌍 Can't believe he's missing out on their lives. Homework's a mess, and I'm doing it all. 😡 When is he ever gonna slow down and be here for us?? 😭💔"



Write the next text message, where she positively describes the CEO physically in details (skin, eyes, hair, ...).



"Despite all the frustration, I can't deny that man's got a magnetic charm that's hard to resist. 🤩 His skin is that perfect sun-kissed bronze, and those deep, piercing blue eyes always get me. ✨ His hair, just the right shade of salt and pepper, makes him look even more distinguished. 😍 Sometimes, it's just hard to stay mad at someone who looks like a movie star. 🌟"

# Course organization



Célia  
Nouri



Francis  
Kulumba



Rian  
Touchent



# Course organization

- Objectives
  - ***Knowlegde*** : Understand how the main components of modern NLP pipelines work.
  - ***Hands-on Practice*** : Develop the ability to build NLP systems for most major NLP tasks (classification, question-answering, ChatGPT-like assistant...).
  - ***Critical Thinking*** : Understand the flaws and challenges of NLP, and come up with creative ideas to improve current systems.

# Course organization

## Part 1: General NLP

- ***What*** : Tokenization, Language Modeling, Small Models, Interpretability
- ***Goal*** : Understand, build and deploy a custom ChatGPT-like assistant.

# Program

- **Session 1 (Today, Célia):** Recap
- **Session 2 (17/10, Francis):** Tokenization
- **Session 3 (24/10, Francis):** Language Modeling
- **Session 4 (31/10, Francis):** Modern NLP with limited resources
- **Session 5 (7/11, Francis):** Modern Interpretability
- **Session 6 (14/11, Francis/Célia):** Midterm project session

# Course organization

## Part 2: Advanced NLP Applications

- ***What*** : Human-Centered, Tasks and Applications, Multilinguality, Multimodality
- ***Goal*** : Understand, build and deploy NLP systems across languages, media, and real-world contexts.

# Program

- **Session 7 (21/11, Célia):** Human-Centered NLP
- **Session 8 (28/11, Rian):** Advanced NLP Tasks
- **Session 9 (5/12, Rian):** Domain-specific NLP
- **Session 10 (12/12, Rian):** Multilingual NLP
- **Session 11 (19/12, Célia):** Multimodal NLP
- **Session 12 (16/01, Francis/Célia/Rian):** Final Presentations!

# Evaluation

- Group project (4-5 people)
- Two options
  - *Demo*
  - *R&D project*



# Evaluation - Demo (option 1)

Use a well-known approach to produce a MVP for an original use-case and present it in a demo.

*Example: An online platform that detects AI-generated text.*

## Evaluation - R&D project (option 2)

Based on a research article, conduct original experiments and produce a report.

*Example: Do we need Next Sentence Prediction in BERT? (Answer: No)*

# Evaluation

- Mid-term project evaluation (14/11, 25%)
  - Project proposal
  - First elements
- Final project (16/01, 75%)
  - Short report showing each person's contribution (by 9/01)
  - Github repo (by 9/01)
  - Oral group presentations

# Evaluation

- You can already constitute teams
- Send an email:
  - By 17/10
  - To {celia.nouri, francis.kulumba, rian.touchent}@inria.fr
  - With the names of team members (also cc'ed)
  - *Demo* or *R&D*
  - Small description of project (few sentences)
- If you really have no idea what to do, ask us

# Questions

# Recap

# Quiz time!

<https://docs.google.com/forms/d/1BZaBagWlpVgKLsT2NdjJ4pXzPXTBnsxv52jEF4CR6GY/prefill>



# Basic concepts in NLP

# Stemming / Lemmatization

**Stemming** shortens variations (*inflected forms*) of a word to an identifiable root

Example:

*Flying using airplanes harms the environment*

=>

*Fly us airplane harm the environ*

# Stemming / Lemmatization

**Lemmatization** groups variations (*inflected forms*) of a word to an identifiable representative word

Example:

*Flying using airplanes harms the environment*

=>

*Fly **use** airplane harm the environ**ment***

# Tokenization

**Tokenization** turns text strings (= lists of characters) into lists of meaningful units (e.g. words or subwords)

Example:

*Flying using airplanes harms the environment*

=>

*( Fly | ing | us | ing | air | planes | harms | the | environ | ment )*

(see Course 2)

# Regular expressions

**Regular expressions** is a string that specifies a match pattern in text

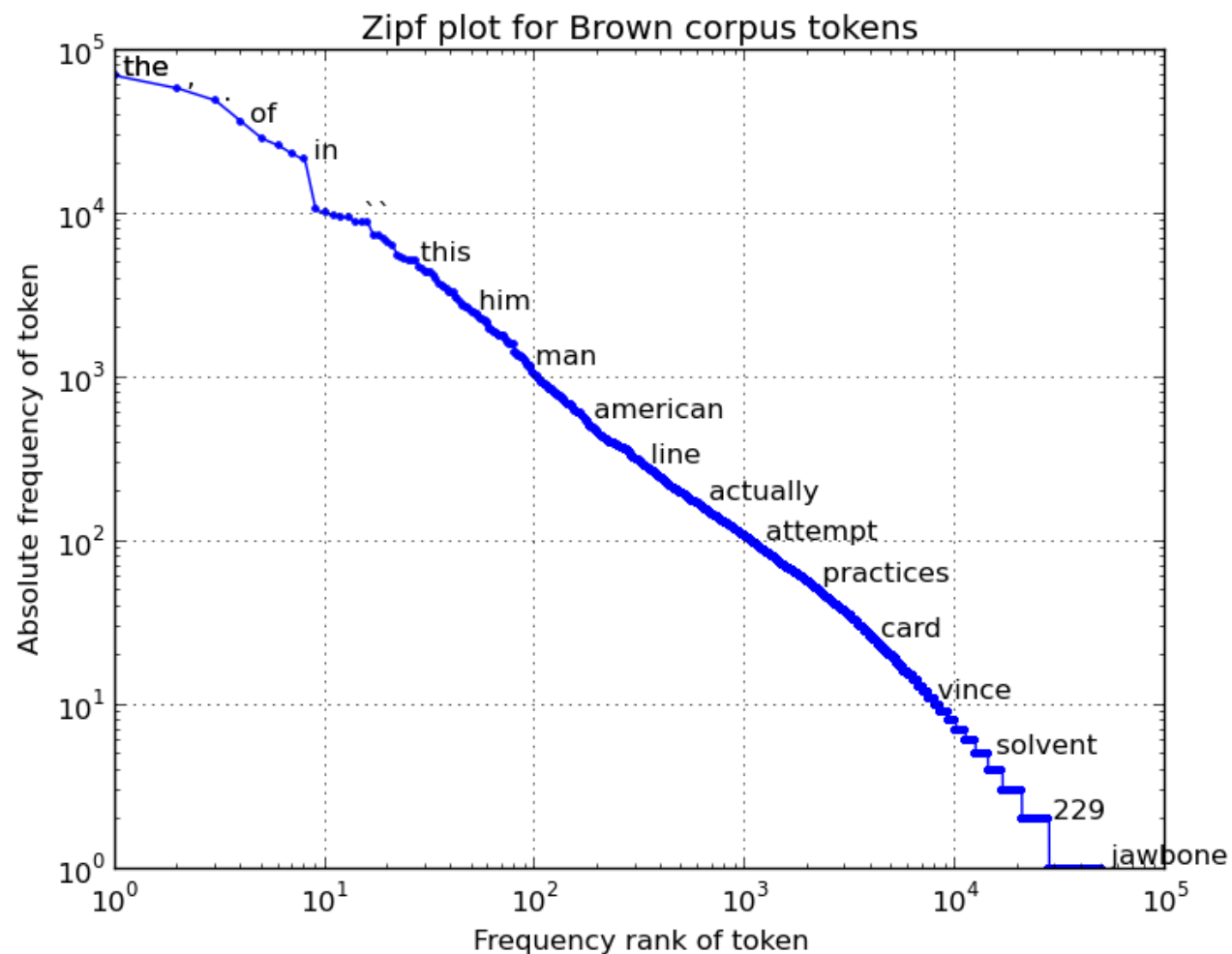
Example:

`/\w*ing\b/` -> *Flying using airplanes harms the environment*  
=

Two matches:

- *Flying*
- *using*

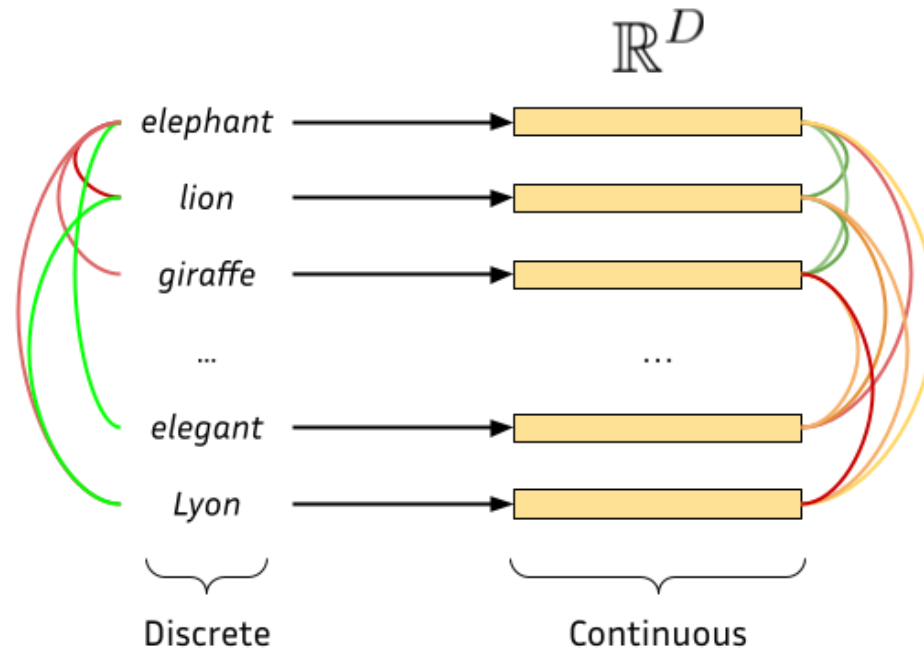
# Zipf's law



# Machine Learning & NLP

# Embeddings

- Vectors that represent textual entities (words, sentences, documents, ...)





# Bag-of-Words Embeddings

Represent a sentence/document by counting words in it.

Example:

*John likes to watch movies. Mary likes movies too.*

=>

```
{John: 1, likes: 2, to: 1, watch: 1, movies: 2, Mary: 1, too: 1}
```

=>

```
[1, 2, 1, 1, 2, 1, 1]
```

# TF-IDF Embeddings

Represent a sentence/document by comparing how frequent a word is in the extract vs. how frequent the word is in general.

$$w_{x,y} = \text{tf}_{x,y} \times \log \left( \frac{N}{\text{df}_x} \right)$$

**TF-IDF**

Term  $x$  within document  $y$

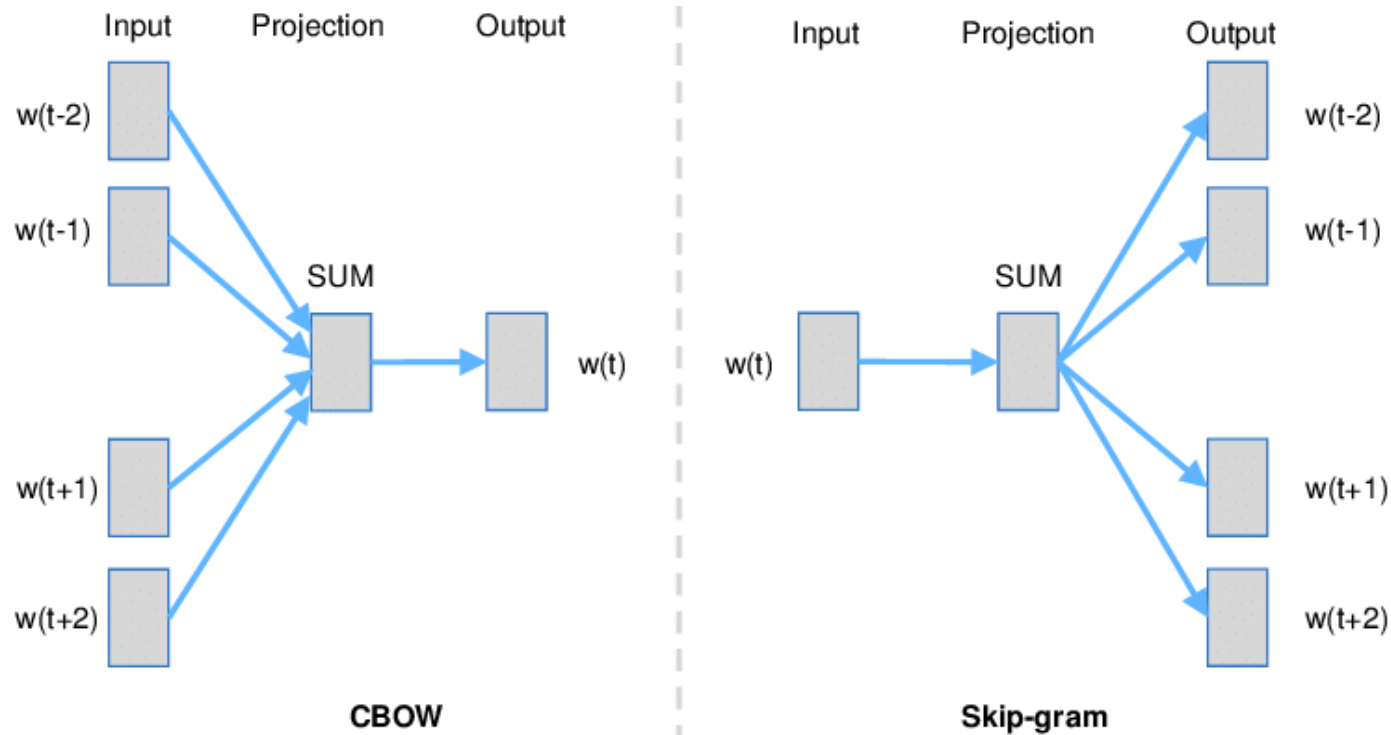
$\text{tf}_{x,y}$  = frequency of  $x$  in  $y$

$\text{df}_x$  = number of documents containing  $x$

$N$  = total number of documents

# Skip-gram & CBoW

Learn embeddings **without supervision/counting**.

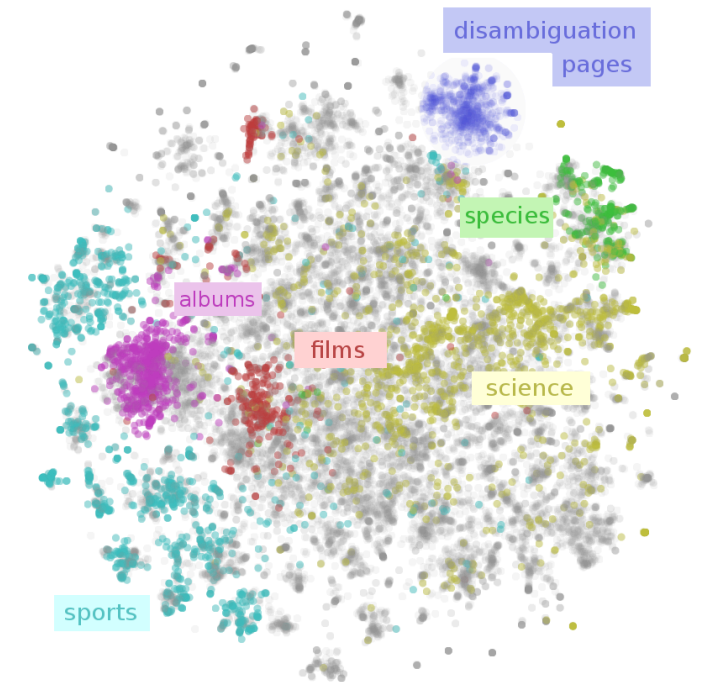


# Static word embeddings

Allow semantically meaningful spaces and can be used as features.

Known static embeddings include:

- GloVe
- FastText
- Word2Vec
- ...

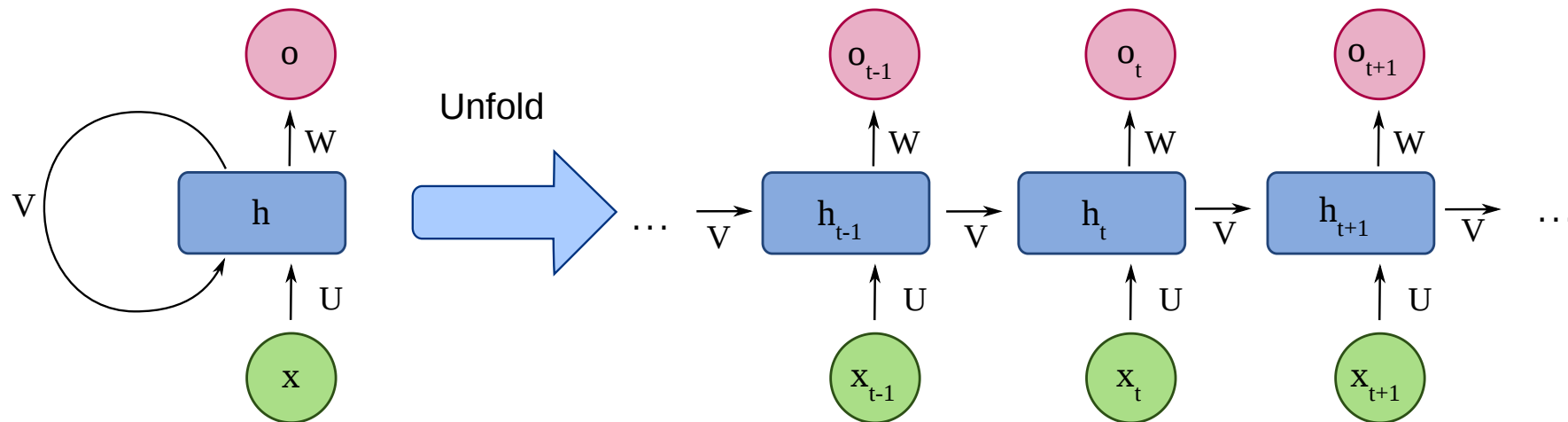


# Deep Learning & NLP

# RNNs

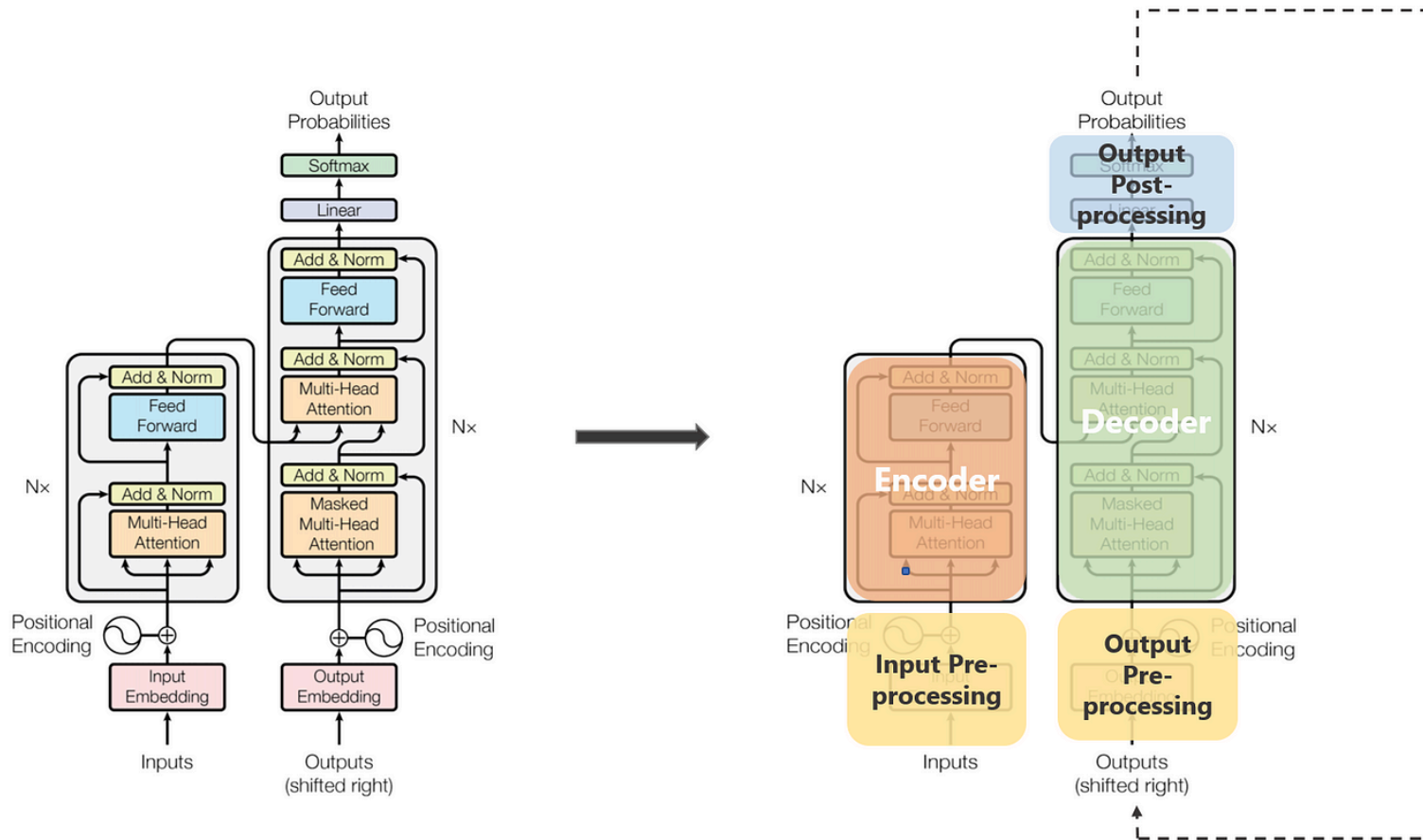
## Recurrent Neural Networks

Neural Networks that are able to process input sequences recurrently.



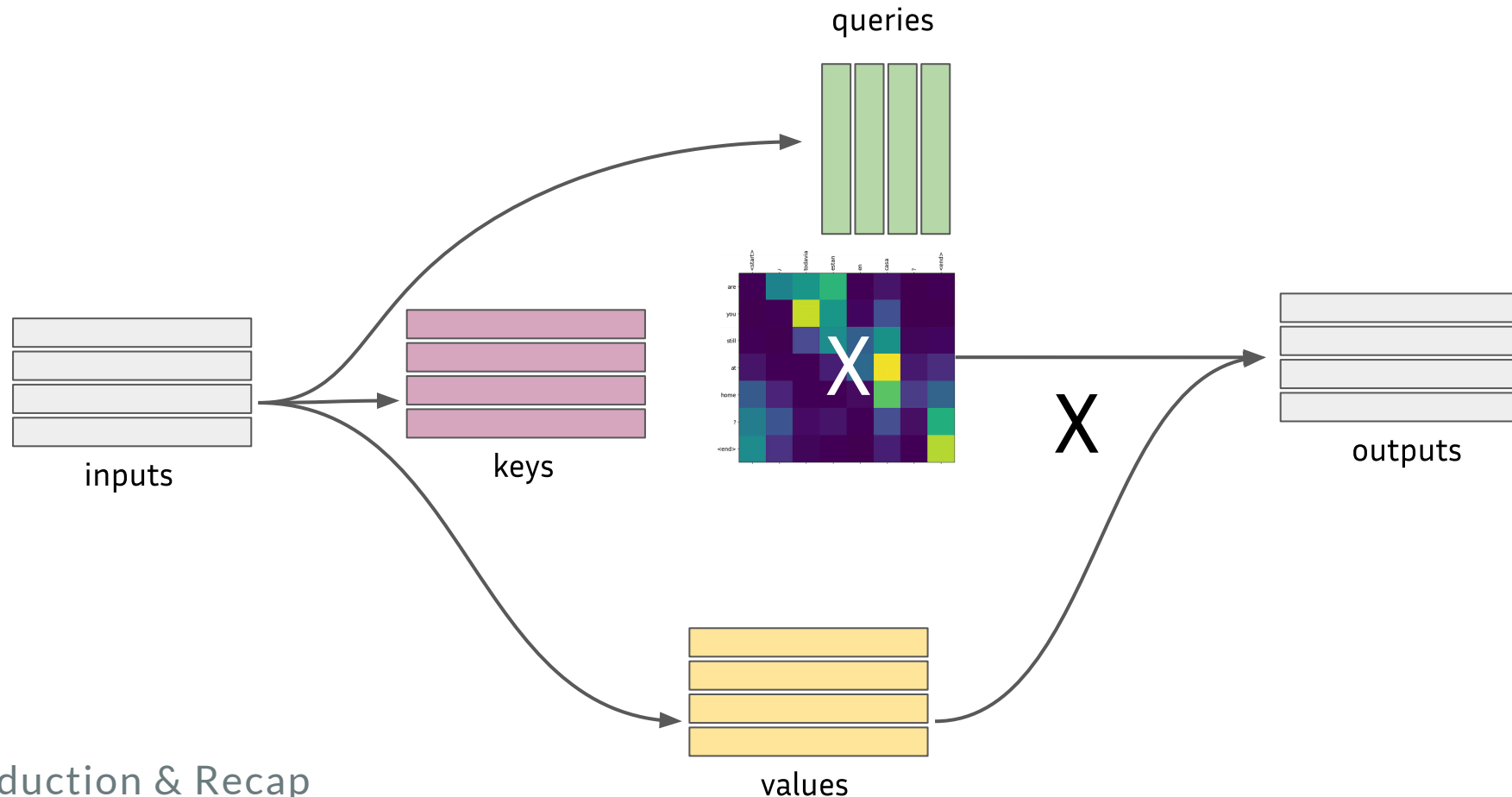
# Transformers

Neural Networks that are able to process input sequences directly.



# Transformers - Self-Attention

Self-attention allows comparing inputs and representing interactions.





# Fine-tuning

Modern NLP models are built in two separate steps:

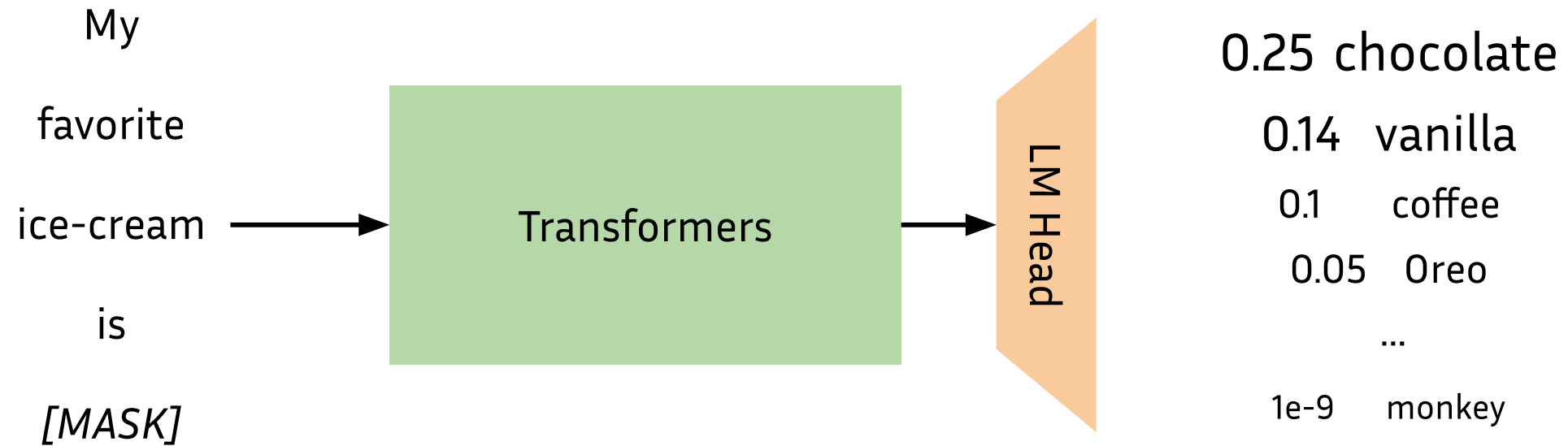
- **Pretraining:** models are trained **without supervision** on raw text data (e.g.: next word prediction on all text from Wikipedia).
- **Fine-tuning:** pretrained models are *retrained* on a smaller annotated dataset that matches the final task.

# Fine-tuning - example

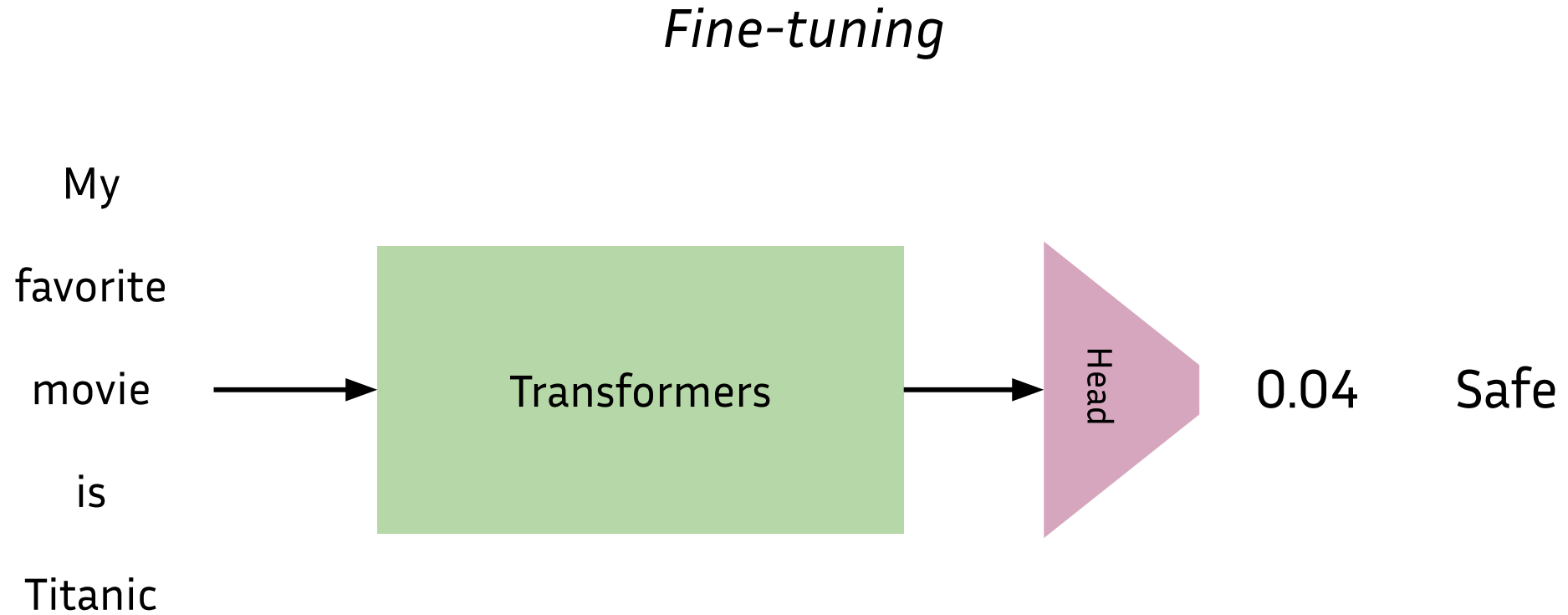
1. Download a Transformers language model that was pretrained on data from the Internet (books, Wikipedia, ...) to predict future words
2. Take a list of tweets and mark them as *suspicious* or *safe*
3. Add a classifier on top of the LM that predicts a float in  $[0, 1]$
4. Train the whole model (LM + classifier) to predict *suspicious* or *safe* tweets

# Fine-tuning - example

*Pre-training*



# Fine-tuning - example



# Questions?

# Lab session